Research Update

Andy

9/20/24

1 Introduction

A lot of NLP today relies on the ability to parallelize computations in order to process large amounts of linguistic data in a reasonable amount of time. As such, people often use parallelizable computational models with bounded depth. What are the implication of this reliance on parallelism?

- Do we lose any important expressive capabilities by only considering highly parallelizable computations?
- On the other hand, does the restriction to constant depth parallel computation make it easier for these models to learn certain important functions?

We study the transformer, one of the most prominent of these parallel architectures. We examine how they process formal languages, idealized model of sequences with unbounded length.

- What formal languages can transformers express?
- What formal languages can transformers learn?

2 Logic as a Model of Parallel Computation

One of the main conceptual tools we use is first-order logic. It is a mathematical formalism where we can make statements like

$\exists x.\mathsf{green}(x)$ There exists a green object

The intuition is that a quantifier $\exists$ is a model of a parallelized search across the entire input. The connections between first-order logic and parallel computation are deep, explored in Immerman (1998). We can consider augmentations and restrictions of logic. Most importantly for us, the ability to count.

$\#_x\mathsf{green}(x) > \#_x\mathsf{red}(x)$ There are more green objects than red objects

We find that transformers, as instances of constant depth parallelizable algorithms, can be closely modeled using logic.

3 RASP as a Better Model Than Logic

All models are wrong, but some are useful

— George E.P. Box

Example 3.1.

$$D := \overleftarrow{\#}\left[Q_(\right] = \overleftarrow{\#}\left[Q_(\right] \wedge \overleftarrow{\#}\left[\overleftarrow{\#}\left[Q_(\right] < \overleftarrow{\#}\left[Q_)\right]\right] = 0$$

Example 3.2.

$C_(i) := \#\left[j \leq i\right]\ Q_(j)$ ▷ The number of (up to position i
$C_)(i) := \#\left[j \leq i\right]\ Q_)(j)$ ▷ The number of) up to position i
$V(i) := C_(i) < C_)(i)$ ▷ *Violation*: there are more) than (
$C_V(i) := \#\left[j \leq i\right]\ V(j)$ ▷ The number of *Violations*
$M(i) := C_V(i) = 0$ ▷ *Matched*: zero *Violations*
$B(i) := C_(i) = C_)(i)$ ▷ *Balanced*: same number of (and)
$D(i) := M(i) \wedge B(i)$ ▷ String is *Matched* and *Balanced*

RASPs are perhaps a more pleasing way to represent these computations than in logic. Furthermore, the way they make the number of variables more explicit is also of technical significance.

4 Extensions of C-RASP

Definition 4.1 (C-RASP)**.** Let Σ be an alphabet.

Boolean-Valued Operations		Count-Valued Operations	
Symbol	$P(i) := Q_\sigma(i)\ \sigma \in \Sigma$	**Counting**	$C(i) := \#\left[j \leq i\right]\ P(j)$
Negation	$P(i) := \neg P_1(i)$	**Conditional**	$C(i) := P(i)\ ?\ C_1(i)\ :\ C_2(i)$
Conjunction	$P(i) := P_1(i) \wedge P_2(i)$	**Addition**	$C(i) := C_1(i) + C_2(i)$
Constant	$P(i) := \top$	**Subtraction**	$C(i) := C_1(i) - C_2(i)$
Modular	$P(i) := MOD_m^r(i)$	**Constant**	$C(i) := 1$
Previous	$P(i) := \ominus P(i)$		
Comparison	$P(i) := C_1(i) \leq C_2(i)$		

Example 4.2. Consider some examples in the following table on the string *abaaba*.

	a	b	a	a	b	a
$Q_a(i)$	$\top$	$\bot$	$\top$	$\top$	$\bot$	$\top$
$Q_b(i)$	$\bot$	$\top$	$\bot$	$\bot$	$\top$	$\bot$
$\#\left[j \leq i\right]\ Q_a(j)$	1	1	2	3	3	4
$\ominus Q_b(i)$	$\bot$	$\bot$	$\top$	$\bot$	$\bot$	$\top$
$MOD_2^0(i)$	$\top$	$\bot$	$\top$	$\bot$	$\top$	$\bot$

5 Expressivity of C-RASP

This formalism can express a lot of things we think transformers do well, such as: Dyck-1, Majority, Induction heads, $(aa)^*$.

Also interestingly, we can also prove that this formalism cannot express many things we think transformers can't do well, such as: Parity, **since** operators, multiplication, Dyck-2

Why should this be the case? It seems that the only non-uniform attention patters which work on unbounded input lengths, that transformers can learn well, are those that correspond to Previous and Modular operations. This is what Michael Hahn has been looking into.

6 C-RASP as a "natural" Class of Languages

If we add future and past counting to C-RASP along with $\ominus$, C-RASP joins the following cohort of equivalent logics

$$\mathsf{FO}^2[<, +1, \#] = \widehat{\mathsf{Maj}}_2[<, +1] = \mathsf{UTL}[\#]$$

Maybe there are other members of this class? Like visibly-counter automata or something. This would be interesting to show that the class of languages is somewhat robust.

References

Neil Immerman. 1998. *Descriptive complexity*. Springer Science & Business Media.